

Tutorial 2 (part 1)

qq. 2.1, 2.2(1-3), 2.5

SC1006, CZ1106

2.1 ARM Addressing Modes, their Characteristics and Applications

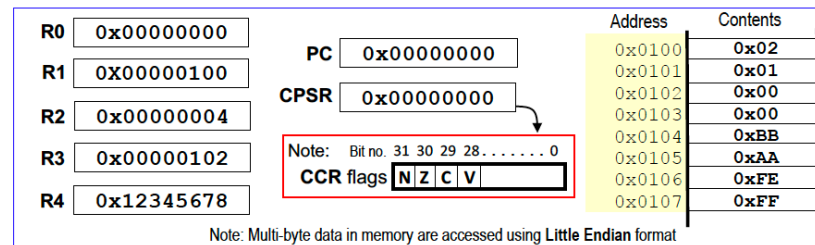


Figure 1 – Initial state of some ARM processor registers and memory contents

- (1) Give the name of the source operand addressing mode for each mnemonic in Table 1.
- (2) Give the 32-bit content in register **R0** immediate after the execution of each instructions in Table 1. (Note: Each execution is independent of the others and uses the same initial values shown in Figure 1 and Little Endian byte-ordering is assumed).

No.	Mnemonics	Source addressing mode	Content in R0
1.	MOV R0, R2		
2.	MOV R0, #0x0100		
3.	LDR R0, [R1]		
4.	LDR R0, [R1, #4]		

1. Understand how the different addressing modes work.
2. Understand when to use which addressing mode. Different addressing modes provide different ways to access operands in register and memory. Under certain circumstance, the right choice can allow more efficient codes to be written.
3. Understand how to sequence different ARM data transfer instructions (with appropriate addressing modes) to achieve specified operation (e.g. swapping registers, memory copy, etc).

- (4) With reference to the three addressing modes (a) to (c) in part (3), describe which addressing mode you would use for the following operations and why:
 - (a) Fetch the next element of an integer array into a register for subsequent comparison.
 - (b) Copy a frequently used 32-bit constant value into another temporary register.
 - (c) Initialise a loop counter register with a small value that does not exceed 255.
- (5) Suggest how you can move the 32-bit content in address location **0x0100** to **0x0104**. You may assume the initial values in the registers are shown in Figure 1.

2.1 ARM Addressing Modes, their Characteristics and Applications

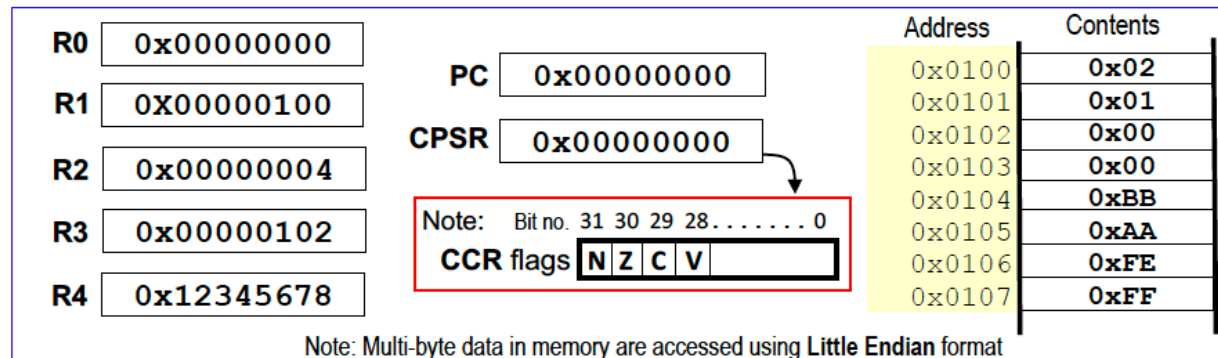
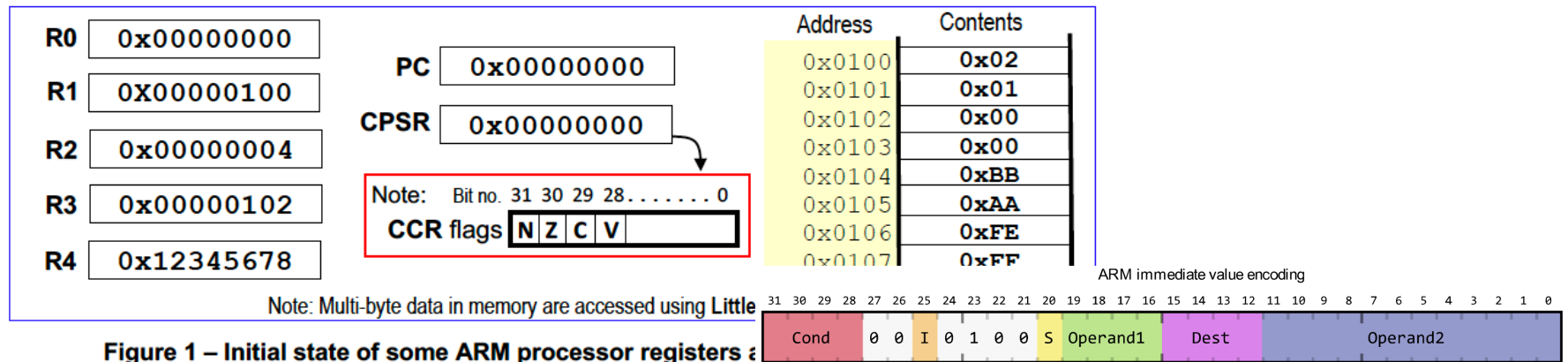


Figure 1 – Initial state of some ARM processor registers and memory contents

- (1) Give the name of the source operand addressing mode for each mnemonic in Table 1.
- (2) Give the 32-bit content in register **R0** immediate after the execution of each instructions in Table 1. (**Note:** Each execution is independent of the others and uses the same initial values shown in Figure 1 and Little Endian byte-ordering is assumed).

No.	Mnemonics	Source addressing mode	Content in R0
1.	MOV R0,R2		
2.	MOV R0,#0x0100		
3.	LDR R0,[R1]		
4.	LDR R0,[R1,#4]		
5.	LDR R0,[R1,R2]		
6.	LDR R0,[R1,#4]!		
7.	LDR R0,[R1],#4		
8.	LDR R0,[R1,R2]!		
9.	LDR R0,[R1],R2		

Table 1 – The different ARM addressing modes and its effects after execution



No.	Mnemonics	Source addressing mode	Content in R0
1.	MOV R0 ,R2	Register Direct	0x00000004
2.	MOV R0 ,#0x0100	Immediate	0x00000100
3.	LDR R0 , [R1]	Register Indirect	0x00000102
4.	LDR R0 , [R1 , #4]	Base plus Offset (Pre-index)	0xFFFEAABB
5.	LDR R0 , [R1 , R2]	Base plus Index Register (Pre-index)	0xFFFEAABB
6.	LDR R0 , [R1 , #4] !	Offset with Autoindexing (Pre-index)	0xFFFEAABB
7.	LDR R0 , [R1] , #4	Offset with Autoindexing (Post-index)	0x00000102
8.	LDR R0 , [R1 , R2] !	Index Register with Autoindexing (Pre-index)	0xFFFEAABB
9.	LDR R0 , [R1] , R2	Index Register with Autoindexing (Post-index)	0x00000102

Note that the source addressing modes from instruction 4 onwards are all effectively register indirect (it is excluded in the naming since it is implicit and this will shorten the name of the AM). Also, for addressing modes with Auto index, the base register R1 is modified after the execution. What is added to the base register R1 can either be a known offset specified using and immediate value (e.g. #4) or the 32-bit content of an Index Register (e.g. R2).

(3) Using the initial values shown in Figure 1, give the ARM instructions that will move the 32-bit value of **0x00000102** into register **R0** using the following addressing modes:

(a) Register direct

(b) Immediate

(c) Register indirect

Note: If necessary, you may use more than one instruction to achieve this requirement.

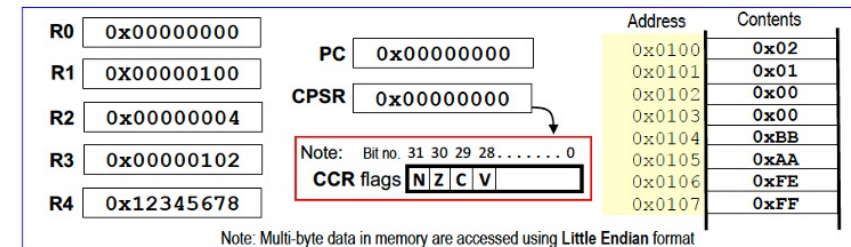


Table 1 – The different ARM addressing modes and its effects after execution

(3) Using the initial values shown in Figure 1, give the ARM instructions that will move the 32-bit value of **0x00000102** into register **R0** using the following addressing modes:

(a) Register direct

(a) Register direct

Suggested solution:

The value **0x102** can be found in register **R3**. We can simply copy the content in **R3** into **R0** using register direct.

MOV R0,R3 ; copy 32-bit content in R3 into R0

(b) Immediate

Suggested solution:

The value **0x102** cannot be directly moved into **R0** using immediate addressing because in the ARM instruction encoding, only 12 bits are allocated to specify the immediate value. So, this value (**I**) is specified with only 8 bits (i.e. 0x00 to 0xFF) and can be shifted anywhere within the 32-bit range using a right shift specified with the remaining 4 bits. The value of these 4-bits (**S**) can left shift the 8-bit value of **I** by (**S** × 2) bits.

MOV R0,#0x100 ; move the value 0x100 into R0

ADD R0,R0,#2 ; add 2 to R0 to get 0x102

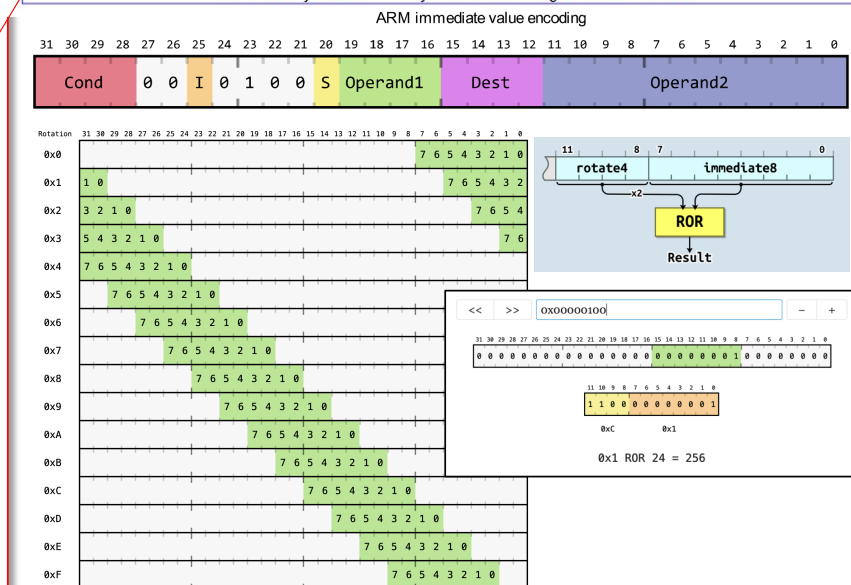
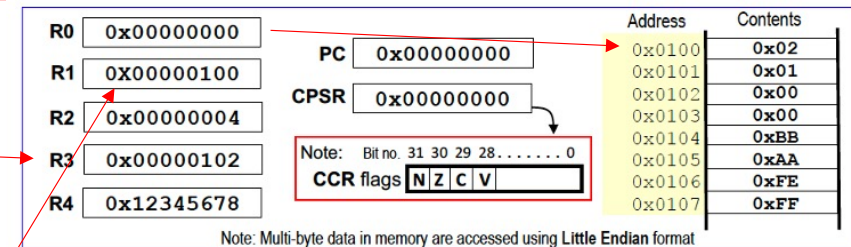
(c) Register indirect

Suggested solution:

Inspection of the memory content in Figure 1 shows that the 32-bit value of **0x00000102** is stored starting at address **0x100**. Since the content in register **R1** is pointing to address **0x100**, it can be used to retrieve the required value from memory using register indirect addressing.

LDR R0,[R1] ; use reg indirect to get 0x102 from mem addr 0x100

(c) Register indirect



<https://alisdair.mcdiarmid.org/arm-immediate-value-encoding/>

- (4) With reference to the three addressing modes (a) to (c) in part (3), describe which addressing mode you would use for the following operations and why:
 - (a) Fetch the next element of an integer array into a register for subsequent comparison.
 - (b) Copy a frequently used 32-bit constant value into another temporary register.
 - (c) Initialise a loop counter register with a small value that does not exceed 255.
- (5) Suggest how you can move the 32-bit content in address location **0x0100** to **0x0104**. You may assume the initial values in the registers are shown in Figure 1.

(4) With reference to the three addressing modes (a) to (c) in part (3), describe which addressing mode you would use for the following operations and why:

- (a) Fetch the next element of an integer array into a register for subsequent comparison.
- (b) Copy a frequently used 32-bit constant value into another temporary register.
- (c) Initialise a loop counter register with a small value that does not exceed 255.

(a) Fetch the next element of an integer array into a register for subsequent comparison.

Suggested solution:

Since the element of an array is stored in memory, a **register indirect** addressing mode with the **LDR** instruction should be used. If these elements are fetched consecutively within a loop, it will be more efficient to use register indirect with Base plus Offset Auto Pre-index (or Post-index), which automatically updates the array pointer register to the next element before (or after) fetching the memory content.

(b) Make a copy of a frequently used 32-bit constant value into a temporary register.

Suggested solution:

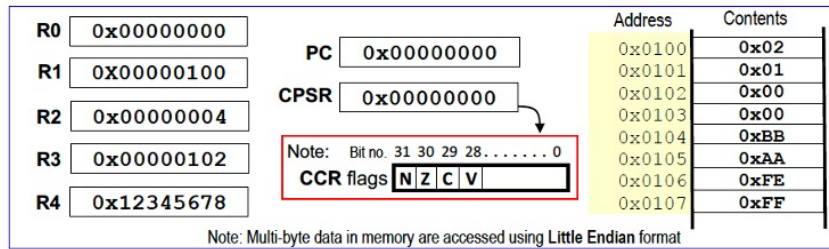
If an arbitrary but frequently used 32-bit value is needed in a program, it would be best to make a copy of it in a spare register and use **register direct** to copy it into a temporary register using the **MOV** operator. Immediate addressing could also be used but remember, there are many 32-bit values that cannot be specified using a single immediate data transfer.

(c) Initialise a loop counter register with a small value that does not exceed 255.

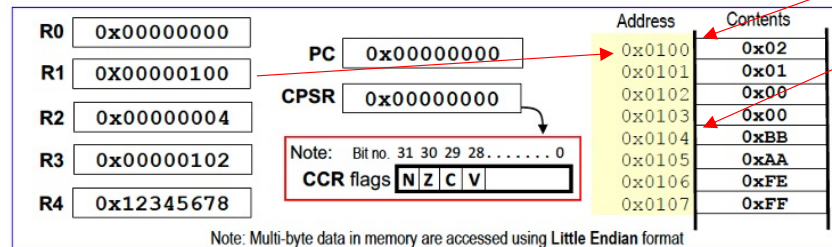
Suggested solution:

To initialise a loop counter register with a small value (i.e. < 256), the easiest method is to use the **MOV** instruction with **immediate** addressing. These small values are always specifiable with immediate addressing.

- (5) Suggest how you can move the 32-bit content in address location **0x0100** to **0x0104**.
You may assume the initial values in the registers are shown in Figure 1.



- (5) Suggest how you can move the 32-bit content in address location **0x0100** to **0x0104**. You may assume the initial values in the registers are shown in Figure 1.



With the ARM ISA, a memory to memory transfer cannot be achieved directly but only with the use of a temporary register.

Based on Figure 1, since the initial value in **R1** is already **0x00000100**, it can be used as a pointer register to retrieve the value at address **0x100** into a temporary register (e.g. **R0**). Then a register indirect with base plus offset of 4 could be used to copy this retrieved value in **R0** into the address **0x104**, as shown:

```
LDR R0 , [R1]           ; move the value in 0x100 into R0
STR R0 , [R1 , #4]       ; move the value in R0 into 0x104
```

- (6) How can the memory copy operation specified in part (5) be done efficiently for 7 memory locations with a loop structure, as shown in Figure 2? Assume the initial values in the registers are shown in Figure 1. Do not concern yourself with the code constructs for the loop structure, only the body of the loop that does the memory copy for contents in **0x0100** to **0x0104** , **0x0104** to **0x0108** and so on.

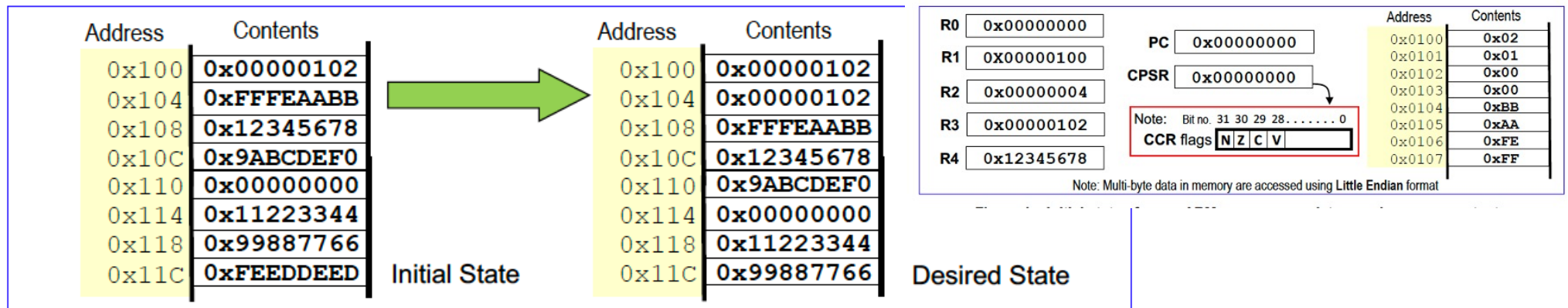


Figure 2 – The initial and desired end result of the memory transfer operation.

- (7) Give the ARM assembly code to swap the 32-bit values in registers **R2** and **R3**. Do the same for the 32-bit values in address locations **0x0100** to **0x0104**?

To be covered only if time permits.

- (8) Describe the characteristics of the **Full Descending** (FD) stack. Give the instruction to push register **R0** to a FD stack which is maintained by the stack pointer (**SP**) and pop a word from a FD stack into register **R1**.
(Note: operations on the FD stack are relevant to the chapter on Modular Programming)

- (6) How can the memory copy operation specified in part (5) be done efficiently for 7 memory locations withing a loop structure, as shown in Figure 2? Assume the initial values in the registers are shown in Figure 1. Do not concern yourself with the code constructs for the loop structure, only the body of the loop that does the memory copy for contents in **0x0100 to 0x0104** , **0x0104 to 0x0108** and so on.

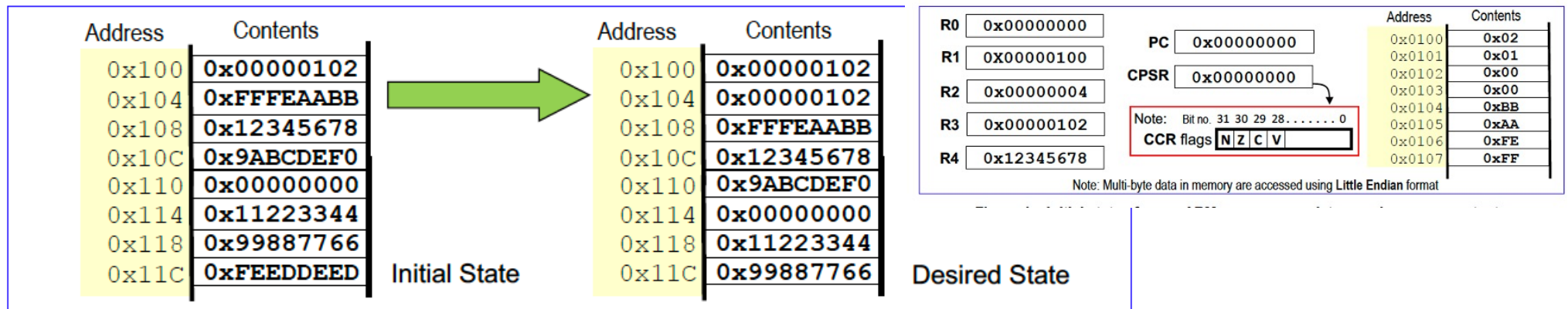
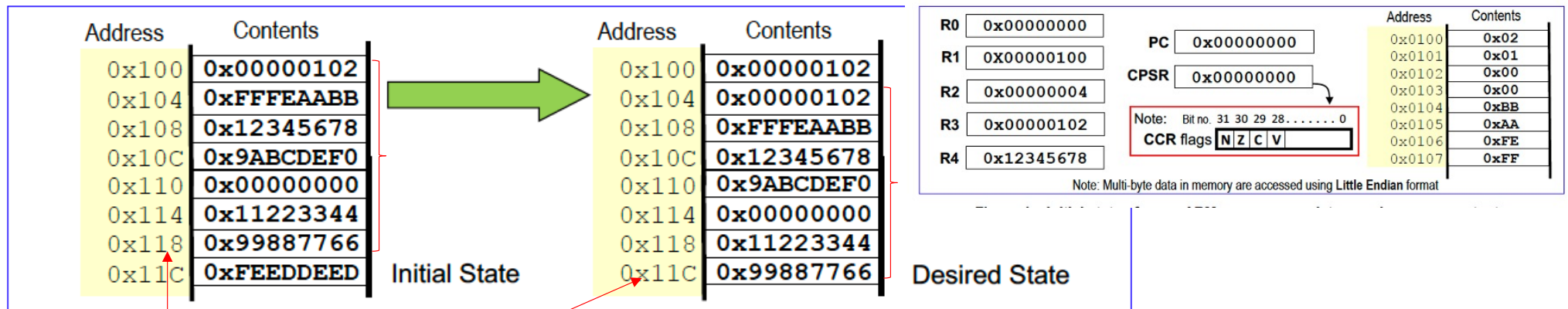


Figure 2 – The initial and desired end result of the memory transfer operation.

- If such operations are repeated over consecutive memory locations, it is more efficient to use some form of auto-indexing with the memory access. This will automatically position the memory pointer to the next place for the subsequent retrieval and storage.
- Copying must start at the higher memory address and work itself toward a lower to avoid writing over memory locations that has yet to be copied to its new location.

- i. If such operations are repeated over consecutive memory locations, it is more efficient to use some form of auto-indexing with the memory access. This will automatically position the memory pointer to the next place for the subsequent retrieval and storage.
- ii. Copying must start at the higher memory address and work itself toward a lower to avoid writing over memory locations that has yet to be copied to its new location.



Method 1:

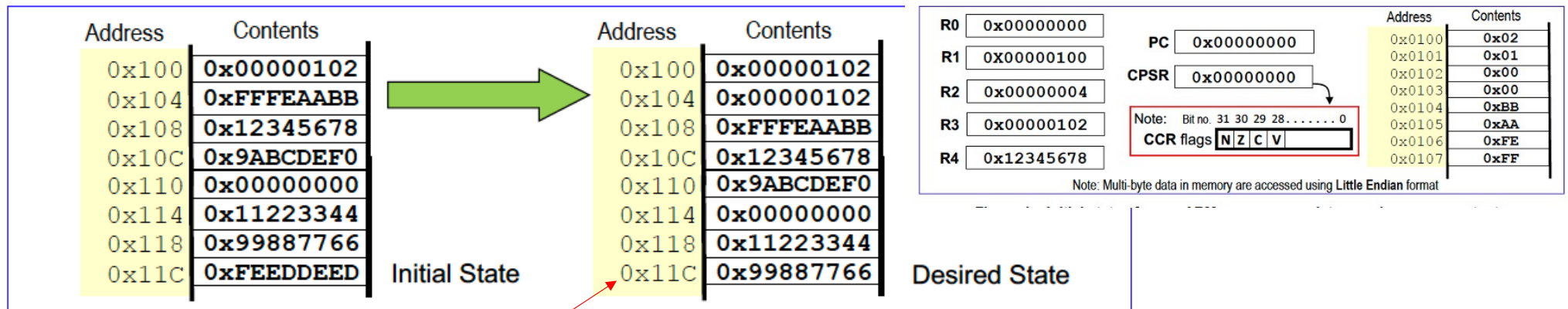
Use two index registers. One to point to the source memory address and another to point to the destination memory address. Start at the highest memory address and keep decrementing these two registers by 4 (size of 32-bit integer) each time around the loop.

```

ADD R3, R1, #0x1C    ; initialise R3 with highest dest address from 0x100
SUB R2, R3, #4        ; initialise R2 with highest source address from 0x100
:                     ; setup loop structure
Loop LDR R0, [R2]      ; move the value in current source address into R0
     STR R0, [R3]      ; move R0 into current destination address
     ADD R3, R3, #-4   ; decrement destination address pointer R3
     ADD R2, R2, #-4   ; decrement source address pointer R2
     :                ; loop back until 7 loops done
  
```


- If such operations are repeated over consecutive memory locations, it is more efficient to use some form of auto-indexing with the memory access. This will automatically position the memory pointer to the next place for the subsequent retrieval and storage.
- Copying must start at the higher memory address and work itself toward a lower to avoid writing over memory locations that has yet to be copied to its new location.

ently for 7
the initial
h the code
ry copy for



Method 2:

Makes efficient use of register indirect with base plus offset (-ve offset) and register indirect with base plus offset auto post-index (-ve index) to retrieve and store memory to appropriate address locations respectively, each time round the loop. Again, the decrement is by 4 each time to handle the size of the 4-byte integer.

Note: The example below makes use of another register **R3** to avoid destroying the base address of memory data in **R1**. If this is not an issue, even **R3** is not needed, just change **R1** to start at address content **0x11C** and use it instead of **R3**.

```

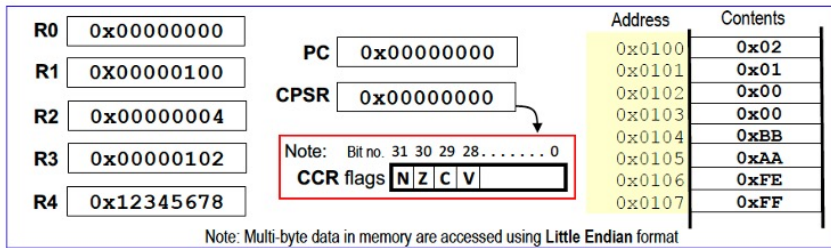
ADD R3,R1,#0x1C ; initialise R3 with highest dest address from 0x100
:               ; setup loop structure
Loop LDR R0,[R3,#-4] ; move the value in current source address into R0
STR R0,[R3],#-4 ; move R0 into current dest address using auto post-index
:               ; loop back until 7 loops done

```

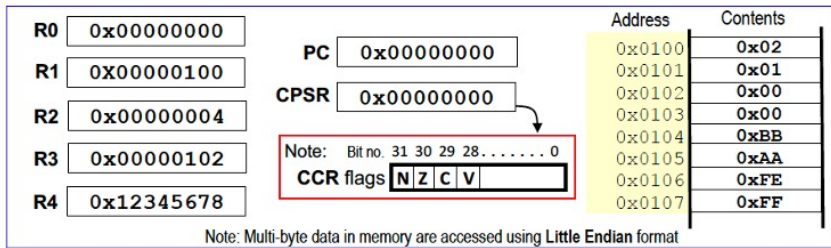
LDR R0,[R1,#4] | Base plus Offset (Pre-index)

LDR R0,[R1],#4 | Offset with Autoindexing (Post-index)

(7) Give the ARM assembly code to swap the 32-bit values in registers **R2** and **R3**. Do the same for the 32-bit values in address locations **0x0100** to **0x0104**?



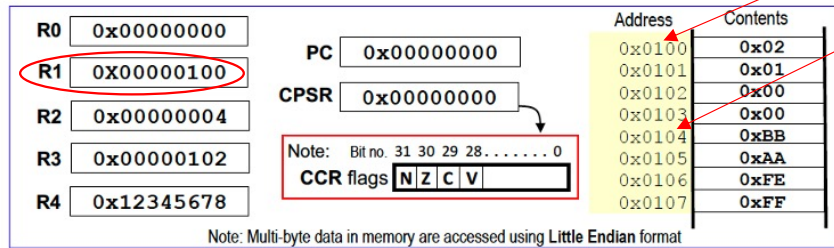
(7) Give the ARM assembly code to swap the 32-bit values in registers **R2** and **R3**. Do the same for the 32-bit values in address locations **0x0100** to **0x0104**?



To swap two registers, an additional temporary register is used. The following code swaps the contents in registers **R2** and **R3**.

MOV R4 ,R2	; move the value in R2 into temp reg R4
MOV R2 ,R3	; Wipe out value in R2 with value in R3
MOV R3 ,R4	; Copy previous value in R2 (that is in R4) into R3

(7) Give the ARM assembly code to swap the 32-bit values in registers **R2** and **R3**. Do the same for the 32-bit values in address locations **0x0100** to **0x0104**?



Now two temporary registers are needed to do the swap because we cannot do a direct memory to memory copy operation.

Assume (see Fig. 1), the address **0x00000100** is currently stored in **R1**. stored starting at address **0x100**. Since the content in register **R1** is pointing to address **0x100**, it can be used to retrieve the required value from memory using register indirect addressing.

```

LDR R2 , [R1]           ; move the value in 0x100 into temp reg R2
LDR R3 , [R1 , #4]      ; move the value in 0x104 into temp reg R3
STR R2 , [R1 , #4]      ; Copy value in 0x104 with value in 0x100 in R2
STR R3 , [R1]           ; Copy value in 0x100 with value in 0x101 in R3
  
```

To be covered only if time permits.

- (8) Describe the characteristics of the **Full Descending** (FD) stack. Give the instruction to push register **R0** to a FD stack which is maintained by the stack pointer (**SP**) and pop a word from a FD stack into register **R1**.

(**Note:** operations on the FD stack are relevant to the chapter on Modular Programming)

To be covered only if time permits.

(8) Describe the characteristics of the **Full Descending** (FD) stack. Give the instruction to push register **R0** to a FD stack which is maintained by the stack pointer (**SP**) and pop a word from a FD stack into register **R1**.

(Note: operations on the FD stack are relevant to the chapter on Modular Programming)

- The Full Descending stack (as the name suggest), grows **towards lower memory address** (descending) each time an item is pushed to the stack.
- The stack pointer (SP) is always **pointing to a valid item** (full) on the top of the FD stack. This means to push an item on the stack, we must first decrement the stack pointer first (pre-index addressing mode must be used).
- To push **R0** to the FD stack, we can use the following instruction:

STR R0 , [SP , #-4] ! ; push R0 to FD stack using auto-indexing (pre-index)

- To pop a 32-bit value from the FD stack into **R1**, we use the instruction:

LDR R1 , [SP] , #4 ; pop FD stack in R1 using auto-indexing (post-index)

Note the negative offset used during a push operation as the FD stack is growing towards lower memory address.

2.2 Arithmetic and Logical Instructions

- (1) Give **four** different ways to **clear** register **R0** to **0x00000000** using only a single ARM mnemonic.
- (2) You are required to **set** bits 0, 4, 8, 12, 24 and 28 in register **R2** while leaving all other bits unchanged. For example, if **R2=0x03468ACF**, then after execution, the new value after execution should be **R2=0x13469BDF**.

Give the sequence of ARM mnemonics to achieve this. See if you can do this using only

1. *Understand that there are maybe many ways to perform the same operations (e.g. clear reg).*
2. *Understand how to design a bit mask for the logical instruction OR to set specific bits in a register.*
3. *Appreciate some applications of the EOR (Exclusive OR operation).*
4. *How shift can be used to implement multiply (divide) by 2^n , where n is the number of bits shifted and the characteristics of overflow when doing signed multiplication*

registers **R2** and **R3** without the use of an additional register.

2.2 Arithmetic and Logical Instructions

(1) Give **four** different ways to **clear** register **R0** to **0x00000000** using only a single ARM mnemonic.

- | | |
|----------------------------|---------------------------------|
| 1. MOV R0 , #0 | (direct data transfer approach) |
| 2. SUB R0 , R0 , R0 | (arithmetic approach) |
| 3. EOR R0 , R0 , R0 | (logical operation approach) |
| 4. AND R0 , R0 , #0 | (logical operation approach) |

(5) With reference to your solution in question 2.1 part (7), did you have to use an additional register to swap the registers **R2** and **R3**? Consider the situation where there are no spare registers left for you to use. How can one of the logical instructions be used to swap the registers **R2** and **R3** without the use of an additional register.

2.2 Arithmetic and Logical Instructions

- (1) Give **four** different ways to **clear** register **R0** to **0x00000000** using only a single ARM mnemonic.
- (2) You are required to **set** bits 0, 4, 8, 12, 24 and 28 in register **R2** while leaving all other bits unchanged. For example, if **R2=0x03468ACF**, then after execution, the new value after execution should be **R2=0x13469BDF**.
Give the sequence of ARM mnemonics to achieve this. See if you can do this using only one additional register besides the target register **R2**.
- (3) With reference to your solution in question 2.1 part (7), did you have to use an additional register to swap the registers **R2** and **R3**? Consider the situation where there are no spare registers left for you to use. How can one of the logical instructions be used to swap the registers **R2** and **R3** without the use of an additional register.

2.2 Arithmetic and Logical Instructions

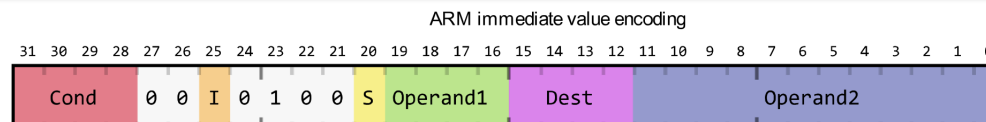
(1) Give **four** different ways to **clear** register **R0** to **0x00000000** using only a single ARM mnemonic.

(2) You are required to **set** bits 0, 4, 8, 12, 24 and 28 in register **R2** while leaving all other bits unchanged. For example, if **R2=0x03468ACF**, then after execution, the new value after execution should be **R2=0x13469BDF**.

Give the sequence of ARM mnemonics to achieve this. See if you can do this using only one additional register besides the target register **R2**.

00010001 00000000 00010001 00010001_b=0x11001111

```
MOV R0, #0x11          ; immediate data cannot to load 0x11001111 into register
ADD R0, R0, #0x1100     ; need to do this in three stages
ADD R0, R0, #0x11000000 ; finally R0 has the value 0x11001111
ORR R2, R2, R0          ; use logical OR with mask in R0 to set specific bits in R2
```



2.2 Arithmetic and Logical Instructions

- (1) Give **four** different ways to **clear** register **R0** to **0x00000000** using only a single ARM mnemonic.

To swap two registers, an additional temporary register is used. The following code swaps the contents in registers **R2** and **R3**.

```
MOV R4 , R2      ; move the value in R2 into temp reg R4
MOV R2 , R3      ; Wipe out value in R2 with value in R3
MOV R3 , R4      ; Copy previous value in R2 (that is in R4) into R3
```

- (3) With reference to your solution in question 2.1 part (7), did you have to use an additional register to swap the registers **R2** and **R3**? Consider the situation where there are no spare registers left for you to use. How can one of the logical instructions be used to swap the registers **R2** and **R3** without the use of an additional register.

2.2 Arithmetic and Logical Instructions

Three exclusive OR operation will allow the content in R2 and R3 to be exchanged without the need of any additional registers. No memory access is needed. The ability of the EOR operation to detect differences between two bits is exploited here.

```
EOR R2 , R2 , R3    ; X := X XOR Y  
EOR R3 , R2 , R3    ; Y := X XOR Y  
EOR R2 , R2 , R3    ; X := X XOR Y
```

- (3) With reference to your solution in question 2.1 part (7), did you have to use an additional register to swap the registers **R2** and **R3**? Consider the situation where there are no spare registers left for you to use. How can one of the logical instructions be used to swap the registers **R2** and **R3** without the use of an additional register.

2.5 Program Counter related Addressing Modes

Figure 4 shows an ARM assembly program and the starting address of various incomplete instructions in code memory. Complete the mnemonics **M1** to **M5** based on their respective comments and ensure that all your solutions support **position-independent code**. You may use the partially completed VisUAL ARM assembly program template *Tutorial 2_5-Template* to test out your answers.

Note: The **PC** points 8 addresses ahead during the execution of each instruction as this is a consequence of how the ARM processes instructions using “pipeline” techniques.

Address	Mnemonics or Hexa Data	Comments
0x000	MOV R0, #0	; Clear R0
0x004	? PC, ?, ?	; M1 - Relative Jump to instruction at address 0x00C (label Addr_0C)
0x008	MOV R1, R0	; Dummy instruction
0x00C	ADD R1, ?, ?	; M2 - Get start address Var_100 into R1 in a PC-relative manner
0x010	LDR R0, ?	; M3 - Move the content at memory variable Var_104 into R0.
0x014	ADD R2, ?, ?	; M4 - Get start address of next instruction into R2.
0x018	MOV ?, ?	; M5 - Create an infinite loop with this instruction.
:	:	
0x100	0x01234567	; Var_100 - Constant stored in data memory at address 0x100
0x104	0x89ABCDEF	; Var_104 - Constant stored in data memory at address 0x104

Figure 4 – Various ARM mnemonics and their respective start addresses in code memory

1. Understand how PC relative addressing can support position independent code.
2. Understand how the modification of PC contents affects program flow.
3. Understand how to compute the offset relative to the PC to access the data memory area

2.5 Program Counter related Addressing Modes

Figure 4 shows an ARM assembly program and the starting address of various incomplete instructions in code memory. Complete the mnemonics **M1** to **M5** based on their respective comments and ensure that all your solutions support **position-independent code**. You may use the partially completed VisUAL ARM assembly program template *Tutorial 2_5-Template* to test out your answers.

Note: The **PC** points 8 addresses ahead during the execution of each instruction as this is a consequence of how the ARM processes instructions using “pipeline” techniques.

Address	Mnemonics or Hexa Data	Comments
0x000	MOV R0, #0	; Clear R0
0x004	ADD PC, PC , #0	; M1 - Relative Jump to instruction at addr 0x00C
0x008	MOV R1, R0	; Dummy instruction
0x00C	ADD R1, PC , #0xEC	; M2 - Get start addr Var_100 into R1 in a PC-relative manner

M1 – When the instruction at address 0x004 executes, the PC will be pointing 8 location ahead. This means it is already pointing to the instruction at 0x00C. In order to make a relative jump to that instruction, we just need to **ADD** zero to the current PC value. Adding to the current PC value is the way to make a relative jump.

M2 – In order to make the code position independent, we must assume that we have no knowledge of where the actual address the data we want to access resides. We can only access the data with the knowledge of its offset from the current PC position. As such, we must employ PC relative addressing. Position independent program (i.e. consisting of both executable code and its associated memory data) can be shifted anywhere in memory and still execute correctly.

The offset is computed by subtracting the address where the data is stored (i.e. 0x100) with the current PC value during the execution of the ADD instruction at 0x00C. Since the PC is always 8 location ahead during execution, the PC value during execution of the ADD instruction is $(0x00C + 0x008) = 0x014$. The PC relative offset to the memory variable at 0x100 is therefore given by $0x100 - 0x014 = \mathbf{0x0EC}$.

0x00C	ADD R1, PC, #0xEC	; M2 - Get start addr Var_100 into R1 in a PC-relative manner
0x010	LDR R0, [R1, #4]	; M3 - Move the content at memory variable Var_104 into R0.
0x014	ADD R2, PC, #-4	; M4 - Get start address of next instruction into R2.
0x018	MOV PC, R2	; M5 - Create an infinite loop with this instruction.
:	:	
0x100	0x01234567	; Var_100 - Constant stored in data mem at address 0x100
0x104	0x89ABCDEF	; Var_104 - Constant stored in data mem at address 0x104

Figure 4 – Various ARM mnemonics and their respective start addresses in code memory

2.5 Program Counter related Addressing Modes

Figure 4 shows an ARM assembly program and the starting address of various incomplete instructions in code memory. Complete the mnemonics **M1** to **M5** based on their respective comments and ensure that all your solutions support **position-independent code**. You may use the partially completed VisUAL ARM assembly program template *Tutorial 2_5-Template* to test out your answers.

Note: The **PC** points 8 addresses ahead during the execution of each instruction as this is a consequence of how the ARM processes instructions using “pipeline” techniques.

Address	Mnemonics or Hexa Data	Comments
0x000	MOV R0 , #0	; Clear R0
0x004	ADD PC , PC , #0	; M1 - Relative Jump to instruction at addr 0x00C
0x008	MOV R1 , R0	; Dummy instruction
0x00C	ADD R1 , PC , #0xEC	; M2 - Get start addr Var_100 into R1 in a PC-relative manner
0x010	LDR R0 , [R1 , #4]	; M3 - Move the content at memory variable Var_104 into R0.

M3 – Since the start address of the memory data area is already in register R1 as a result of the instruction at M2, we need to add a positive offset of 4 to the base address in R1 in order to get to the next 32-bit memory variable Var_104. We can do this using register indirect with base plus offset.

2.5 Program Counter related Addressing Modes

Figure 4 shows an ARM assembly program and the starting address of various incomplete instructions in code memory. Complete the mnemonics **M1** to **M5** based on their respective comments and ensure that all your solutions support **position-independent code**. You may use the partially completed VisUAL ARM assembly program template *Tutorial 2_5-Template* to test out your answers.

Note: The **PC** points 8 addresses ahead during the execution of each instruction as this is a consequence of how the ARM processes instructions using “pipeline” techniques.

Address	Mnemonics or Hexa Data	Comments
0x000	MOV R0, #0	; Clear R0
0x004	ADD PC, PC, #0	; M1 - Relative Jump to instruction at addr 0x00C
0x008	MOV R1, R0	; Dummy instruction
0x00C	ADD R1, PC, #0xEC	; M2 - Get start addr Var_100 into R1 in a PC-relative manner
0x010	LDR R0, [R1, #4]	; M3 - Move the content at memory variable Var_104 into R0.
0x014	ADD R2, PC, #-4	; M4 - Get start address of next instruction into R2.

M4 – In order to get address of the next instruction, we need to remember that the PC will be 8 addresses ahead during the execution of the instruction. So making a copy of the PC will not get us the next instruction but the address of two instructions away. In order to get the address of the immediate next instruction, we will need to subtract 4 from the current PC value. The ADD instruction is used to add -4 to the current PC value before moving the total summed value into destination register R2.

2.5 Program Counter related Addressing Modes

Figure 4 shows an ARM assembly program and the starting address of various incomplete instructions in code memory. Complete the mnemonics **M1** to **M5** based on their respective comments and ensure that all your solutions support **position-independent code**. You may use the partially completed VisUAL ARM assembly program template *Tutorial 2_5-Template* to test out your answers.

Note: The **PC** points 8 addresses ahead during the execution of each instruction as this is a consequence of how the ARM processes instructions using “pipeline” techniques.

Address	Mnemonics or Hexa Data	Comments
0x000	MOV R0 , #0	; Clear R0
0x004	ADD PC , PC , #0	; M1 - Relative Jump to instruction at addr 0x00C
0x008	MOV R1 , R0	; Dummy instruction
0x00C	ADD R1 , PC , #0xEC	; M2 - Get start addr Var_100 into R1 in a PC-relative manner
0x010	LDR R0 , [R1 , #4]	; M3 - Move the content at memory variable Var_104 into R0.
0x014	ADD R2 , PC , #-4	; M4 - Get start address of next instruction into R2.
0x018	MOV PC , R2	; M5 - Create an infinite loop with this instruction.

M5 – In order to get an instruction to execute in an infinite loop, it has to jump back to itself. Since we have instruction M4 to load the start address of the next instruction into R2, we can make the next instruction jump back to itself by making it load the value in R2 into the PC. Both instruction M4 and M5 are position independent since no knowledge of the actual address where these instructions were stored is actually employed.